

Praktikum Pemrograman Berorientasi Objek

Pertemuan 5

Materi Pertemuan 5

Gambaran Materi Pembelajaran Hari Ini

Materi Pertemuan 5

01

UML

Memahami perancangan sistem berorientasi objek menggunakan
Unified Modeling Language

02

Recalling

Mengingat kembali dan mengimplementasikan materi PBO yang
telah dipelajari

UML (Unified Modeling Language)

Definisi

UML adalah salah satu "bahasa" standar yang digunakan untuk memodelkan visualisasi, perancangan, dan pendokumentasian aplikasi.

"Bahasa" di sini bukanlah bahasa pemrograman, melainkan berupa **simbol grafis, bangun datar, dan garis koneksi** yang standar.

Analogi Blueprint

UML ibarat sebuah *blueprint* (cetak biru) untuk membangun gedung.

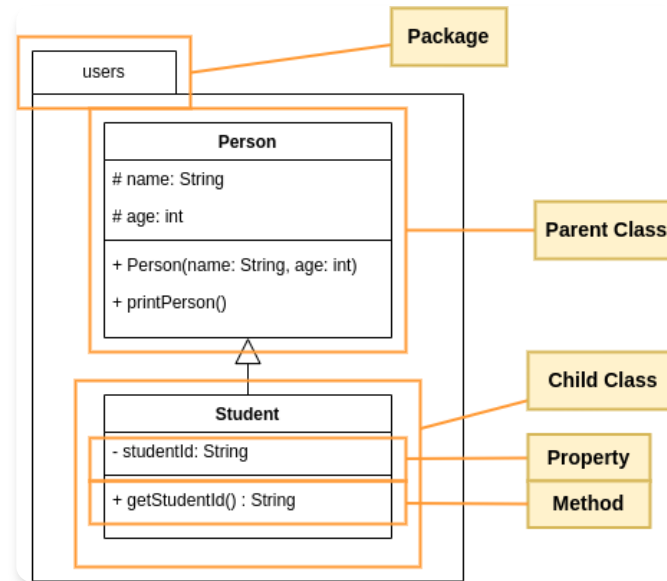
Bangunan yang kompleks tentu membutuhkan *blueprint* yang berbeda-beda tergantung fungsinya (misal: denah per lantai, skematik jalur pipa, atau rute evakuasi). Inilah alasan mengapa UML memiliki **banyak tipe diagram** sesuai tujuannya.

Tipe-tipe Diagram UML:

- **Class diagram** (paling sering) ★
- Use case diagram
- Composite structure diagram
- State machine diagram
- Sequence diagram
- Collaboration diagram
- Deployment diagram
- *dan lain-lain...*

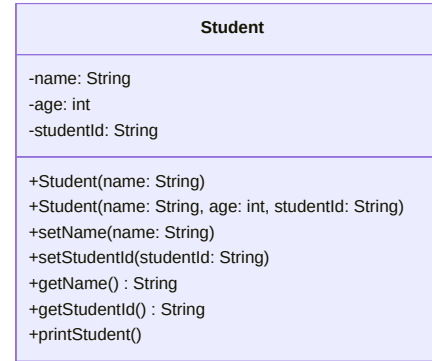
Class Diagram

- Salah satu tipe UML yang sering digunakan adalah class diagram.
- Class diagram menunjukkan struktur dari suatu sistem, seperti kelas, method, atribut, dan hubungan di antaranya.
- Class diagram membantu kita untuk memahami bagaimana sistem diatur dan bagaimana interaksi antar komponen terjadi.



Contoh kelas `Student` dan bagaimana menggambarannya di diagram:

```
class Student {  
    private String name;  
    private int age;  
    private String studentId;  
  
    public Student(String name) { ... }  
    public Student(String name, int age, String studentId) { ... }  
  
    public void setName(String name) { ... }  
    public void setStudentId(String studentId) { ... }  
  
    public String getName() { ... }  
    public void printStudent() { ... }  
}
```



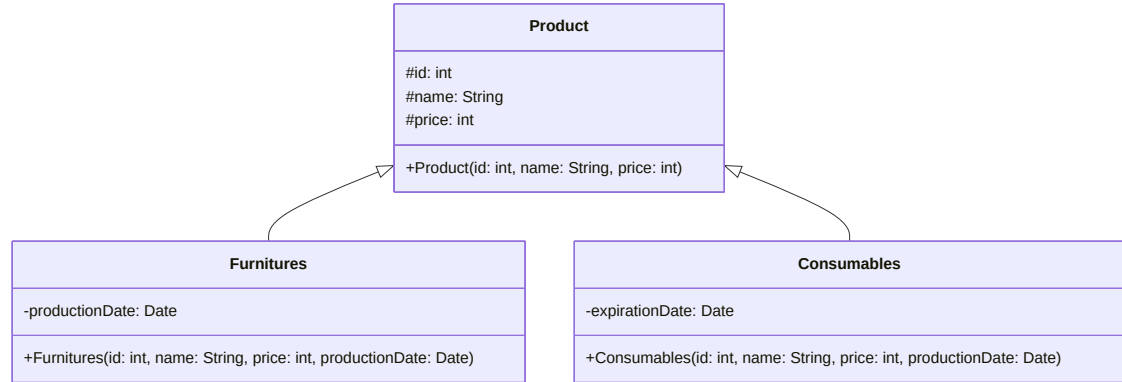
Visibilitas Class Diagram

Cara menentukan access modifier/visibilitas dari atribut dan method pada class diagram UML:

Simbol	Access Modifier
+	public
-	private
#	protected
~	package

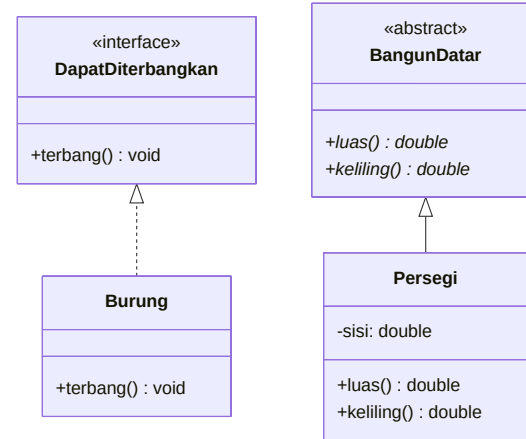
Hubungan Antar Kelas

Inheritance / Pewarisan: membuat *subclass* yang mewariskan sifat-sifat dari *class* yang sudah ada. Memiliki hubungan wajib is-a.



UML Interface & Abstract Class

- **Abstract class** dituliskan dengan nama class yang dibuat *Italic* (miring).
- **Interface** dituliskan dengan penanda `<<interface>>` di atas nama kelasnya.
- Garis penyambung implementasi *interface* menggunakan garis putus-putus.



OOAD

- **Object-Oriented Analysis and Design (OOAD)** adalah pendekatan desain software yang memodelkan sistem menggunakan objek dunia nyata.
- OOAD menggabungkan prinsip Object-Oriented Analysis (OOA) dan Object-Oriented Design (OOD).
- Contoh: Di sistem manajemen bioskop, objek seperti `Film`, `Customer`, `Studio`, dan `Petugas` berinteraksi mengatur penayangan.

Aspek-aspek penting dalam OOAD:

- **Object-Oriented Programming:** direpresentasikan sebagai objek dengan atribut dan method.
- **Pola Desain:** blueprint *reusable* struktur class.
- **Diagram UML:** merepresentasikan interaksi sistem.
- **Use Cases:** membantu mengerti dan mendesain kebutuhan sistem.

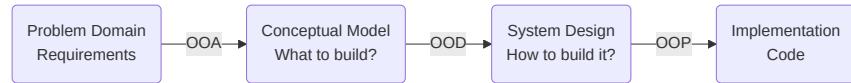
OOA

- **Object-Oriented Analysis** adalah proses memahami kebutuhan sistem dengan mengidentifikasi objek dunia nyata, atribut, perilaku, dan relasinya (tanpa implementasi).
- Contoh: Di sistem bioskop, OOA menentukan apa saja yang Customer dan Petugas lakukan (Customer membeli tiket, Petugas menerima uang).
- OOA memecah masalah jadi *to-do list*: apa yang diketahui sistem, dilakukan sistem, dan perilaku tiap objek.

 OOAD Diagram

OOD

- **Object-Oriented Design** membantu mengubah konsep-konsep dalam OOA menjadi desain mendetail yang bisa diimplementasikan dalam kodingan.
- OOD bertindak sebagai jembatan untuk menghubungkan pemahaman akan masalah yang ada dan pembangunan solusinya.



Keuntungan OOAD

- **Modularitas dan maintainability:** mendorong kita untuk membuat bagian-bagian kecil yang bisa dipakai berulang kali untuk digabungkan dengan sistem yang kompleks.
- **Abstraksi tinggi:** mempermudah pemahaman dan pemeliharaan sistem karena memberikan gambaran yang abstrak dan jelas.
- **Code reuse dan quality:** mendorong penggunaan kembali objek untuk mengurangi duplikasi kode dan meningkatkan kualitas software.
- **Kolaborasi yang lebih baik:** menggunakan bahasa dan pendekatan yang sama sehingga membantu suatu tim untuk berkomunikasi secara lebih efektif.
- **Skalabilitas dan Adaptabilitas:** membangun software yang bisa berkembang dan beradaptasi dengan kebutuhan pengguna yang terus berubah.

Kekurangan OOAD

- **Kompleksitas Sistem:** OOAD bisa sangat kompleks dan membingungkan untuk proyek yang kecil.
- **Kurva Belajar:** Membutuhkan pemahaman mendalam tentang prinsip desain berorientasi objek yang bisa jadi sulit bagi pemula.
- **Over-engineering:** Terkadang pendekatan ini bisa mengarah pada penciptaan kelas dan objek yang tidak perlu, sehingga membuat sistem lebih berat.
- **Waktu Desain:** Membutuhkan banyak waktu di awal untuk analisis dan desain sebelum penulisan kode dimulai.

Recalling Materi

Review Konsep & Sintaks PBO (Minggu 1 - 4)

Recalling: Struktur Class & Encapsulation

Class & Object: Class adalah blueprint, object adalah realisasinya (via `new`). Atribut/properti mendefinisikan *state*, method mendefinisikan *behavior*.

Encapsulation: Praktik menyembunyikan detail internal class (Data Hiding) lewat *access modifier* `private`. Interaksi dari luar hanya diizinkan via `public` method (Getter/Setter).

```
public class User {  
    // Properti (private untuk Encapsulation)  
    private String username;  
    private String password;  
  
    // ... Constructor & Method ...  
}  
  
// Instansiasi Object  
User user1 = new User();
```

Recalling: Constructor & Getter-Setter

Constructor: Dipanggil otomatis saat objek dibuat. Digunakan untuk inisialisasi awal.

Getter & Setter: Method standar untuk mengambil nilai (`get`) dan mengubah nilai (`set`) atribut *private* secara aman (*Data Integrity*).

```
public class User {  
    private String password;  
  
    // Constructor  
    public User(String password) {  
        this.password = password; // 'this' merujuk ke atribut  
    }  
  
    // Setter (Bisa ditambahkan validasi)  
    public void setPassword(String password) {  
        if(password.length() > 5) this.password = password;  
    }  
  
    // Getter  
    public String getPassword() {  
        return this.password;  
    }  
}
```

Recalling: Inheritance & Overriding

Inheritance (`extends`): *Subclass* (anak) mewarisi *Superclass* (induk).

`super` : Keyword pemanggil konstruktor atau method milik induk.

Overriding (`@Override`): *Subclass* menimpa paksa logika method bawaan dari parent-nya.

```
class Hewan {
    public Hewan() { System.out.println("Hewan dibuat"); }
    public void bersuara() { System.out.println("..."); }
}

class Kucing extends Hewan { // Inheritance
    public Kucing() {
        super(); // Memanggil constructor parent
    }

    @Override // Menimpa method parent
    public void bersuara() {
        System.out.println("Meow!");
    }
}
```

Recalling: Kata Kunci this & super

Kata Kunci `this` : Berfungsi untuk merepresentasikan (menunjuk) atribut atau *method* milik instansiasi objek dari kelas itu sendiri. Biasanya digunakan untuk membedakan antara variabel milik kelas dengan *parameter* lokal.

Kata Kunci `super` : Berfungsi untuk merepresentasikan (menunjuk) atribut atau *method* dari *parent class*. Pada *constructor subclass*, `super()` digunakan untuk memanggil dan memberikan data ke *constructor parent*.

```
public class Character {
    protected String name;

    public Character(String name) {
        // 'this' membedakan atribut dengan parameter lokal
        this.name = name;
    }
}

public class Warrior extends Character {
    private double hp;

    public Warrior(String name, double hp) {
        // 'super' mengirim name ke constructor parent
        super(name);
        this.hp = hp;
    }

    public void printStats() {
        // 'super' juga bisa panggil method/atribut parent
        System.out.println("Nama: " + super.name);
        System.out.println("HP : " + this.hp);
    }
}
```

Recalling: Method Overloading

Method Overloading: Mekanisme menulis ulang method dengan **nama yang sama**, tetapi **berbeda parameter** (jumlah, tipe data, atau urutan). Implementasinya bisa berbeda.

Compiler akan otomatis menentukan fungsi mana yang paling cocok dengan inputan.

(Catatan: TIDAK BOLEH overload hanya berdasarkan perbedaan return type)

```
public class Kalkulator {  
  
    // Method 1: Dua parameter int  
    public int tambah(int a, int b) {  
        return a + b;  
    }  
  
    // Method 2: Tiga parameter int (Overloaded)  
    public int tambah(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Method 3: Parameter double (Overloaded)  
    public double tambah(double a, double b) {  
        return a + b;  
    }  
}
```

Recalling: Abstraksi (Abstract & Interface)

Abstract Class: Kelas yang tak dapat diinstansiasi langsung via `new`. *Abstract Method* dideklarasikan tanpa *body/implementasi*. *Subclass* wajib meng-*override*-nya.

Interface (implements): Berbentuk abstrak implisit. Ibarat sebuah **Contract** yang wajib dipatuhi. Semua method otomatis memiliki akses modifier `public`.

```
// Interface
interface BisaTerbang {
    void terbang(); // otomatis public abstract
}

// Abstract Class
abstract class Burung {
    public abstract void bersuara(); // abstrak, no body
    public void makan() { System.out.println("Makan"); }
}

// Implementasi
class Elang extends Burung implements BisaTerbang {
    @Override public void bersuara() { System.out.println("Kwakk"); }
    @Override public void terbang() { System.out.println("Terbang") }
}
```

Recalling: Polimorfisme

Polimorfisme: Kemampuan untuk mempunyai beberapa bentuk class yang berbeda. Terjadi saat suatu objek bertipe **parent class**, tetapi **pemanggilan constructornya melalui child class**.

Contohnya `Kendaraan bmw = new Mobil();` Satu referensi array parent (`Kendaraan[]`) dapat menampung campuran berbagai objek *child* nyata.

```
public class Main {  
    public static void main(String[] args) {  
  
        // Referensinya 'Burung' (Parent), Objeknya 'Elang' (Child)  
        Burung burungSatu = new Elang();  
  
        // Referensinya 'Burung' (Parent), Objeknya 'Bebek' (Child)  
        Burung burungDua = new Bebek();  
  
        // Array Polimorfisme  
        Burung[] koleksi = { burungSatu, burungDua };  
  
        for(Burung b : koleksi) {  
            b.bersuara(); // Panggilan polimorfik  
        }  
    }  
}
```

Latihan Minggu 5

Semua bisa karena terbiasa

Instruksi Latihan

Tugas Minggu 5

Mari kita asah kemampuan kita lebih dalam lagi!

Instruksi Tugas

Teknis Pengumpulan

Terima Kasih!

Do you have any questions? Please use respective class discussion channel on Discord.

Key of success adalah kunci kesuksesan 🔥🔥🔥